

What problem are we solving?

Imagine you run a company with 500 computers. Bad things happen on computers every day — someone logs in at 3am, a program tries to write to a sensitive file, a script runs that shouldn't be running. How do you know about it? How do you find it in time?

You need a system that:

- Watches everything happening on every computer
- Collects all that information in one place
- Automatically identifies suspicious activity
- Alerts a security analyst
- Lets the analyst investigate and document what happened

That system is called a SIEM. We built one.

The city analogy — understand the whole thing first

Think of your Windows laptop as a city. Things happen in that city all day — people walk around, cars drive, buildings get accessed. Most of it is normal. Some of it is suspicious.

Our SIEM is like this:

Sysmon is the CCTV camera network inside the city. It watches everything — every person who enters a building (process creation), every door that opens (registry change), every phone call made (network connection). Without cameras, you are blind. With cameras, you see everything that happened.

Wazuh Agent is the security guard who watches the camera feeds and reports back to headquarters. The guard doesn't analyze anything — he just collects what the cameras see and ships it to the security center.

Port Proxy is the road between the city (your Windows machine) and the security headquarters (WSL2 and Docker). WSL2 lives in its own little world with its own addresses. The port proxy is the bridge that lets the guard's reports cross over.

Wazuh Manager is the security headquarters. It receives all the reports, runs them through a rulebook of 3000+ known suspicious patterns, and says "this one is normal, this one is

suspicious, this one is an attack." It also maps every suspicious thing to the MITRE ATT&CK framework — which is basically a catalog of every known hacking technique in existence.

Logstash is the filing clerk at headquarters. Raw reports come in as messy unstructured text. Logstash reads them, pulls out the important fields, labels them properly, and files them in a consistent format. Without this, everything is a blob of text you can't search properly.

Elasticsearch is the filing cabinet — but a very smart one. It stores every single security event and can search through millions of them in milliseconds. When an analyst says "show me every login from an unusual IP in the last 24 hours," Elasticsearch finds it instantly.

Kibana is the analyst's dashboard — the screen on the wall at headquarters showing maps, charts, and tables of everything happening in real time. The analyst looks at Kibana to understand the security posture of the whole environment.

TheHive is the incident report book. When something suspicious is confirmed, a case gets created. The analyst writes down what they found, what they did about it, and closes it with a resolution. Every compliance framework requires this paper trail.

Atomic Red Team is the fire drill. Instead of waiting for a real attack to see if your alarms work, you simulate attacks yourself and verify the alarms fire correctly. It is how you prove your SIEM works instead of just assuming it does.

Now let's talk about the IP addresses — why did we assign fixed ones?

This is a really important concept. Let me explain it with a simple analogy.

You have five friends. You need to call them every day. You have two options:

Option A — your friends keep changing their phone numbers every day. Every day you have to go find out their new number before you can call them. Painful.

Option B — you give each friend a permanent number that never changes.

That's exactly why we gave each container a fixed IP address.

When Kibana needs to talk to Elasticsearch, it needs to know where Elasticsearch lives. If we let Docker assign random IPs, Kibana's config file would have the wrong address the next day. By fixing the IPs we guarantee that Elasticsearch is always at 172.20.0.10, Logstash is always at 172.20.0.11, and so on.

The 172.20.0.0/24 part means we created a private neighborhood — like a private street that only our containers live on. Nothing outside Docker can access this street directly. It is isolated and secure.

Why Docker? Why not just install everything directly?

Imagine you want to set up five different offices — Elasticsearch, Logstash, Kibana, Wazuh, TheHive. Each office needs its own specific version of software, its own specific settings, and they should not interfere with each other.

Installing everything directly on one computer is like trying to run five different businesses from one desk. They would fight over resources, versions would conflict, and fixing one would break another.

Docker gives each service its own isolated container — like its own separate room with its own walls, its own supplies, its own environment. They can talk to each other through the network we defined, but they cannot mess with each other's internals.

Docker Compose is just the blueprint that describes all five rooms, how they connect, and how to set them all up at once. One command — `docker compose up` — and the whole city springs to life.

Why WSL2? Why not just run Linux on its own?

You are on Windows. Most of these tools are Linux-native — they were built for Linux servers, not Windows desktops. WSL2 gives you a full real Linux kernel running inside Windows without needing a separate machine or dual boot.

Think of it like having a room inside your house that is completely set up like a professional kitchen — separate rules, separate equipment, separate environment — but you can walk through the door from your living room any time.

The flow of one security event — from start to finish

Let me walk you through what happens the moment something suspicious occurs on your Windows machine.

You run a PowerShell command that writes to a registry key. Here is exactly what happens next, step by step:

Sysmon sees it immediately. It knows that reg.exe was spawned by PowerShell, what command was run, and which registry key was touched. It writes this as an event to the Windows Event Log.

Wazuh Agent is constantly watching the Windows Event Log including the Sysmon channel we added. It reads this new event within seconds and packages it up as a structured message.

Wazuh Agent tries to send this to 127.0.0.1 on port 1514. The Windows Port Proxy intercepts this and forwards it to the WSL2 IP on the same port. Docker receives it and delivers it to the Wazuh Manager container at 172.20.0.14.

Wazuh Manager receives the event. It runs it through its rulebook. It finds a match — rule 92041 — Base64-like pattern in registry modification. It assigns this a severity level of 10. It maps it to MITRE ATT&CK techniques T1027 and T1112, tactic Defense Evasion. It creates an alert JSON document.

Wazuh Manager ships this alert JSON to Elasticsearch at 172.20.0.10 on port 9200. Elasticsearch stores it in today's index — wazuh-alerts-4.x-2026.05.04.

Kibana which is connected to Elasticsearch picks this up and it appears on the security dashboard within seconds.

Wazuh integratord simultaneously sees a level 10 alert has been generated. It calls our custom Python script. The script calls TheHive's API at 172.20.0.15 on port 9000. TheHive creates a new case automatically — with the title, description, MITRE tags, severity, and analyst assignment all populated.

The SOC Analyst opens TheHive, sees the new case, opens the Kibana dashboard to pull the raw evidence, investigates, documents findings in the case tasks, and closes it resolved.

Total time from attack execution to case in TheHive: under 60 seconds.

The MITRE ATT&CK framework — what is it really?

Think of it like a crime manual — a catalog that lists every known technique criminals use to break into systems. It is organized by tactic (what the attacker is trying to achieve) and technique (how they do it).

Tactics are the goals: Initial Access, Persistence, Privilege Escalation, Defense Evasion, Discovery, Lateral Movement, Command and Control, Exfiltration.

Techniques are the methods: T1078 is using a valid account. T1547.001 is adding to a registry Run key for persistence. T1027 is obfuscating code with Base64.

When Wazuh tags an alert with T1547.001 — Defense Evasion, it is not just telling you something bad happened. It is telling you exactly what stage of an attack this is and what the attacker is trying to achieve. That context is what lets a SOC analyst prioritize correctly.

Why did we choose Wazuh over Elastic Fleet Server?

Fleet Server is like hiring a manager whose only job is to manage the security guards. They don't actually do any security work themselves — they just keep track of where the guards are.

Wazuh is like hiring a security manager who also comes with a 3000-page rulebook of known criminal behaviors, a compliance checklist, a file tamper detection system, and an automatic threat classifier. Same cost. Far more capability.

In a home lab trying to demonstrate real SOC capability, Wazuh gives you security content immediately. Fleet Server gives you agent management and nothing else.

The ports — why these specific numbers?

Ports are like apartment numbers in a building. The building is your computer. Different services live at different apartment numbers.

Port 9200 is where Elasticsearch lives. Every service that wants to store or retrieve data knocks on apartment 9200.

Port 5601 is where Kibana lives. You open your browser and go to that apartment to see the dashboards.

Port 1514 is where the Wazuh Manager's data receiver lives. Agents knock on 1514 to deliver their security events.

Port 1515 is the registration desk. When an agent first connects, it goes to 1515 to introduce itself and get its ID card.

Port 9000 is TheHive. Our integration script knocks on 9000 to create cases.

These numbers are not random. They are the standard defaults for each tool — chosen by the tool developers and documented publicly. Using the defaults makes troubleshooting easier because documentation and community answers all assume the defaults.

What Sysmon actually changed in our lab

Before Sysmon we detected 1 MITRE technique — T1078, just Windows logon events.

After Sysmon we detected 11 MITRE techniques including T1027, T1105, T1547.001, T1059, and more.

Why such a big difference? Standard Windows Event Logs only record high-level things — someone logged in, a service started, an application crashed. They do not record which process created which other process, what command line arguments were used, or what registry key was modified by what program.

Sysmon fills exactly those gaps. It captures the full story — not just that something happened, but exactly how it happened and who was responsible.

Summary — what you built and what it means

You built a complete security monitoring pipeline. Raw events enter at one end from your Windows machine. By the time they exit at the other end they are classified alerts in a case management system with MITRE ATT&CK tags, severity levels, and analyst documentation.

You validated that it works by simulating real attacks and watching the detections fire.

You did this with entirely free open source tools on a personal laptop. The same architecture — with bigger hardware and some production hardening — runs in real company SOC's protecting thousands of endpoints.

That is the context. Now when someone asks you in an interview "how does a SIEM work" — you don't recite a definition. You explain the pipeline. You talk about log normalization. You mention the MITRE ATT&CK mapping. You describe the case management workflow. You explain what Sysmon adds to detection coverage.

Author: Muhammad Usman Saeed